**BLACK PEN** Answer all questions first. **GREEN PEN** Then: tick and rewrite mistakes.

# LESSON 010 · Iteration in C#

Name: _____ Class: _____ Date: _____

Work through the pages in order. Answer in BLACK PEN first. Check in GREEN PEN only after the Exit Ticket.

## Do Now · Recall previous learning

Q1. What is the difference between = and == in C#?

Q2. Give one reason Unicode may be chosen instead of standard ASCII.

Q3. Convert 01011000 0011 to denary using an 8-bit two’s-complement mantissa and a 4-bit two’s-complement exponent.

Q4. For mark = 85 , what does an if (mark >= 80) ... else if (mark >= 50) structure output, and why?

**Do Now** Recall previous learning | **I Do** Read new knowledge | **We Do** Practise with guidance | **You Do** Work independently | **Review & Improve** Check understanding

**BLACK PEN** Answer all questions first. **GREEN PEN** Then: tick and rewrite mistakes.

# I Do · Read new knowledge

## Key vocabulary

Recall the term — Iteration

Recall the term — Count-controlled iteration

Recall the term — Condition-controlled iteration

Recall the term — Counter

Recall the term — for loop

Recall the term — while loop

Recall the term — do...while

## New learning

Iteration repeats instructions — Iteration means executing a block of instructions repeatedly. The loop needs a rule that determines whether another repetition happens.

for = count-controlled — CODE:
for (int i = 0; i < 5; i++)
{
Console.WriteLine(i);
} — The header sets the counter, tests whether the loop continues, then changes the counter after each repetition.

while = test before repeating — A while loop is useful when you do not know how many attempts will be needed. — CODE:
while (valid == false)
{
// ask again
}

do...while = run first, test after — CODE:
do
{
Console.Write("Password: ");
answer = Console.ReadLine();
}
while (answer != "computer"); — The body executes before the condition is checked, so it always runs at least once.

## Worked example

**BLACK PEN** Answer all questions first. **GREEN PEN** Then: tick and rewrite mistakes.

**2. Write a complete new answer in GREEN PEN. Show working if needed:**





**BLACK PEN** Answer all questions first. **GREEN PEN** Then: tick and rewrite mistakes.

Build a times-table program

1. Read the number — CODE:
Console.Write("Number: ");
int number =
int.Parse(Console.ReadLine()); — This happens once before the repetition begins.

2. Count from 1 to 5 — CODE:
for (int i = 1; i <= 5; i++)
{
// repeated instruction
} — i = 1 starts the counter; i <= 5 keeps the loop running; i++ adds 1 after each repetition.

3. Calculate each multiple — CODE:
for (int i = 1; i <= 5; i++)
{
int answer = number * i;
Console.WriteLine(answer);
} — The same two instructions run five times, but i changes each time.

4. Trace a known input — CODE:
number = 7

i = 1 → 7
i = 2 → 14
i = 3 → 21
i = 4 → 28
i = 5 → 35

## We Do · Practise with guidance

Output the squares $1^2$ to $5^2$

Use one for loop. Each repetition should output the square of the current counter.

Complete the pattern

```
for (int i = ?; i <= ?; i++)
{
    Console.WriteLine(?);
}

for (int i = 1; i <= 5; i++)
{
```

```
    Console.WriteLine(i * i);
}
```

Validate 1 to 5

Keep asking until the user enters a number from 1 to 5 inclusive.

What controls the loop?

Password with do...while

Ask at least once, then continue until the answer is "computer" .

Reverse the OCR stopping condition

A do...while runs the body before checking the condition.

Describe iteration precisely

An OCR question asks how iteration is used in a function. A strong answer refers to the specific loop , the repeated lines and the condition that keeps it running.

Describe this loop

```
while (number > 0)
{
    number = number / 2;
}

while (number > 0)
{
    number = number / 2;
}
```

A precise answer names the construct, the repeated code and the continuation condition.





Console.WriteLine("Accepted");
}
}
Key idea: the Boolean remains false until the accepted condition is met.

Q4. Password attempts
CODE:

```
using System;

public class Program
{
public static void Main()
{
string answer;
int attempts = 0;

do
{
Console.Write("Password: ");
answer = Console.ReadLine();
attempts++;
}
while (answer != "computer");

Console.WriteLine("Attempts: " + attempts);
}
}
```

Key idea: do...while is suitable because the password must be asked at least once. The counter increases on every attempt.

### Exit Ticket · Model answer

False. A while loop checks its condition before each iteration and may run zero times.

## Try the question again

Choose one question you got wrong. All correct? Choose the hardest.

**1. Copy the whole original question:**

```
Console.Write("Enter a whole number: ");
int number = int.Parse(Console.ReadLine());

for (int i = 1; i <= 10; i++)
{
int answer = number * i;
Console.WriteLine(answer);
}
}
}
```

Why it works: the loop runs exactly ten times and the counter provides the multiplier.

Q3. Validated age
CODE:

```
using System;

public class Program
{
public static void Main()
{
bool valid = false;
int age = 0;

while (valid == false)
{
Console.Write("Enter an age from 0 to 120: ");
age = int.Parse(Console.ReadLine());

if (age >= 0 && age <= 120)
{
valid = true;
}
else
{
Console.WriteLine("Invalid - try again.");
}
}
```





## You Do · Work independently

BLACK PEN: Write your own answers. Do not look at the Review & Improve pages yet.

### Question 1

Countdown
Use a for loop to output:
10, 9, 8, ... 1

### Question 2

Times-table generator
Ask for a whole number and output its first 10 multiples.

### Question 3

Validated age
Keep asking for an age until the user enters a value from 0 to 120 inclusive.
When valid, output Accepted .

### Question 4

Password attempts
Use do...while to keep asking until the password is "computer" .
Count how many attempts were made and output the total after the correct password.

## Exit Ticket · Answer before checking

A while loop always executes at least once.







## Review & Improve · GREEN PEN

Now change to GREEN PEN. Find each original question. Tick correct answers; read the model and rewrite mistakes.

### Page 1 onwards · Do Now

Q1. = assigns a value. == compares two values for equality.

Q2. Unicode supports a much larger range of characters and writing systems.

Q3. 5.5 Mantissa 01011000 = 0.6875 ; exponent 0011 = 3 ; $0.6875 \times 2^3 = 5.5$ .

Q4. Distinction Conditions are tested top to bottom. The first true branch runs, so the later branch is skipped.

### You Do pages · Independent questions

Q1. Countdown
CODE:

```
using System;

public class Program
{
public static void Main()
{
for (int i = 10; i >= 1; i--)
{
Console.WriteLine(i);
}
}
}
```

Why it works: the counter starts at 10, continues while it is at least 1, and decreases by 1 after each repetition.

Q2. Times-table generator
CODE:

```
using System;

public class Program
{
public static void Main()
{
```